Nbody: keep state local across all steps

The selected flat_pow kernel runs in 143.3736875 ms per 20,000-step unit versus 227.19575 ms upstream: 36.89% less runtime, or 1.5846x speedup. It keeps the original force calculation and update order and matches both energy and all 30 state components bit-for-bit.

The workload simulates the Sun, Jupiter, Saturn, Uranus and Neptune with pairwise gravity. Each timestep visits ten unordered pairs, updates the two velocities for each pair, then advances all five positions. With dt=0.01, one unit contains 200,000 pair interactions and 100,000 body-position updates. Sixteen units accumulate state across 320,000 steps in each measured process; they do not reset between units.

Structure	Contents and use
BODIES	Dictionary of body name to (position list, velocity list, mass).
SYSTEM	Five body records; each position and velocity has three float components.
PAIRS	Ten references to the same body records in deterministic upstream order.
Numeric state	30 position/velocity values plus five masses; Python boxed floats, not a large array.

The upstream timed loop calls report_energy before and after every advance. Momentum offset is performed before its timer; validation of the optimized result is a separate gate. Standard Python arithmetic and pyperf are used, with no NumPy, alternative integrator or native numerical library in the final software.

Final source: variants/bm_nbody_upstream_opt.py, kernel=flat_pow. Latest nbody pair is from 20 September; the later raytrace-only run did not rerun nbody.

Repeated container work dominates the opportunity

The course VM runs release CPython 3.10.12 for clean timing. Each arm has 11 fresh processes, identical work per process and GC disabled. perf stat, native perf record with python3-dbg, cProfile and the official pyperformance baseline execute separately. Only the unprofiled timer decides the 7% runtime target. Configuration details remain in the run manifests rather than dominating this report. Nbody uses 16 accumulated units per timed process and five units for native sampling.

The retained upstream sources are byte-identical to the pyperformance 1.14.0 wheel, independently checked during packaging. Baseline wrappers call upstream benchmark functions directly; optimized wrappers temporarily replace selected methods/functions and then call the same upstream workload. This is a custom comparison harness around genuine pyperformance code. The official suite run is a separate upstream-only cross-check, not a claim of optimized-suite measurement.

Baseline evidence	Interpretation
526 / 1,175 evaluator self samples (44.77%)	Bytecode and operand/reference handling remain substantial.
206 / 1,175 samples containing PyObject_GetItem or PyObject_SetItem	Inclusive union, counted once per stack: repeated generic item access is a clear target.
Ten pairs repeated 20,000 times	Repeated tuple/list unpacking and velocity list stores amplify small per-interaction overheads.
Float boxing/recycling and libm pow symbols	Numeric work remains, but a single symbol share is not a release-time optimization ceiling.

The small physical state easily fits in cache, but Python list access still requires indexed object operations and reference handling. Keeping values in locals removes these operations even when the accesses would have hit cache. The proposed mechanism is fewer interpreter operations and container accesses, not a larger or faster hardware cache.

cProfile places nearly all timed work in advance, but cannot expose each arithmetic operation inside its loop. Native perf samples and source-level work counts are more useful here than total Python call counts, which include wrapper/import differences.

From grouped reuse to flat_pow

The first grouped kernel keeps the first body's velocity components in locals across consecutive partners, then writes them back at the group boundary. It preserves pair order and reduces logical state accesses, but re-enters groups and reloads state each timestep. Its historical 3.40% runtime reduction was below the target.

flat_pow extends the lifetime of local state across the entire advance call. It loads each of the 30 position/velocity components once, binds the five masses, executes all timesteps with scalar local names, and writes the state back once on successful return. The ten pair updates and five position updates are explicitly expanded in upstream order.

Per-step source work	Upstream	Grouped	flat_pow
Component reads	150	114	30 per call
Component writes	75	57	30 per call
Pair traversal	10 loop iterations	4 ordered groups	10 explicit blocks
Numeric-result expressions	270	270	270
Magnitude	dt*d2**(-1.5)	Same	Same

The access counts describe source-level numeric components, not machine loads. In flat_pow the 30 reads and 30 writes are outside the timestep loop; validation and writeback remain part of timed advance. At 20,000 steps, moving even small repeated costs outside that loop removes a large amount of bookkeeping.

flat_pow incorporates and extends the reuse idea behind grouped; it does not call grouped and does not add its speedup on top. flat_unrolled describes the same implementation structure, not an additional optimization switch. Python local variables still reference float objects; arithmetic creates/recycles boxed results.

Preserve the exact update sequence

```
# One of ten explicitly expanded pair blocks
dx = x0 - x1
dy = y0 - y1
dz = z0 - z1
mag = dt * ((dx*dx + dy*dy + dz*dz) ** (-1.5))
b0m = m0 * mag
b1m = m1 * mag
vx0 -= dx * b1m
vy0 -= dy * b1m
vz0 -= dz * b1m
vx1 += dx * b0m
vy1 += dy * b0m
vz1 += dz * b0m
# Finish all pairs before updating any positions.
x0 += dt * vx0  # likewise y/z and the other four bodies
```

Multiplication grouping matters: dx*(m1*mag) is retained rather than reassociated as (dx*m1)*mag. Pair order matters because a body's velocity is updated by several interactions. Position updates occur only after all pairs in the timestep. The power expression is retained, so this optimization does not depend on accepting numerical drift.

The specialization checks canonical five-body, distinct-list float inputs, ordered pairs, float dt and nonnegative integer step counts. It preserves the identities of the original state lists. Delayed writeback preserves successful-call results but not partial list updates if an exception interrupts an arbitrary call; that boundary is stated explicitly.

The saved correctness gate runs 16 x 20,000 steps from matching initial state, compares finite energy and all 30 components as binary floating-point, and obtains energy -0.1690801979358367 with every bit identical. Unit tests also exercise repeated calls, input restrictions, state identity and detection of one-bit corruption. Energy alone would be too weak a correctness check.

Alternatives considered and the selected contract

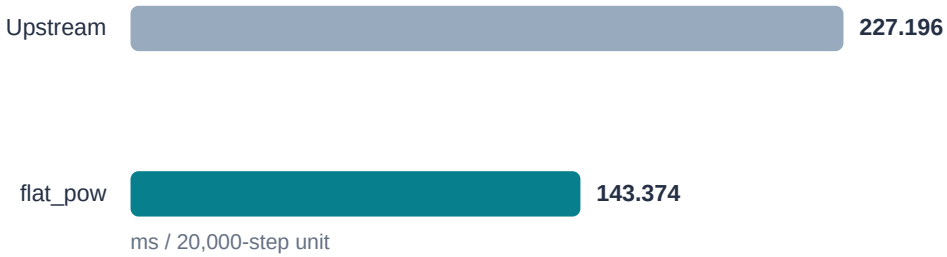
Idea	Decision and reason
Grouped local reuse	Implemented intermediate. A 3.40% historical gain motivated keeping state local for a longer scope.
flat_sqrt / sqrt rewrite	Optional only. Uses dt/(d2*sqrt(d2)); saved full-batch relative energy difference 2.364e-14 and max state difference 2.522e-10. It is not bit-identical.
Barnes-Hut or fast multipole	Not selected for five bodies: overhead and a different force-evaluation path conflict with this exact small workload.
Fewer steps or another integrator	Rejected: changes the requested computation and trajectory.
Tiny NumPy operations	Not implemented as the final path: conversion/call overhead and arithmetic ordering require separate evidence.
Compiled native kernel	Useful separate hardware/software control experiment; not the source of the submitted pure-Python timing gain.

The exactness decision is consistent across the project. A tiny energy difference can be acceptable for a different numerical contract, but it is not equivalent to matching upstream bits. The final software therefore remains flat_pow. Optional hardware RMAG uses the square-root formulation and is labelled separately, rather than silently replacing the selected algorithm.

The initial focus on making pow cheaper addressed a small part of the execution machinery. The successful change removes repeated container accesses, unpacking and loop traversal while leaving arithmetic intact. This is an evidence-supported explanation of the measured improvement, not a claim that arithmetic can never be worth optimizing.

Latest nbody timing: 36.89% less runtime

Per work unit	Upstream	Selected optimized
Median (ms)	227.196	143.374
Mean (ms)	227.906	143.807
Sample SD (ms)	1.990	1.441
Minimum (ms)	226.222	142.481
Maximum (ms)	232.022	146.232
Coefficient of variation	0.873%	1.002%



Unprofiled process medians, normalized by the identical batch size in both arms. Profiler elapsed times are excluded.

The median saving is 83.8220625 ms per 20,000-step unit. The 95% independent-arm bootstrap interval is 35.92%-37.45% runtime reduction, well above the 7% target within the observed process variability. A preceding flat_pow run reported 37.53%; that historical difference alone does not establish a code improvement or a systematic change.

The separate official pyperformance baseline has 60 values and reports approximately 229 +/- 3 ms. Its result is an independent harness cross-check. The final speedup uses the matching custom baseline/optimized pair with identical 16-unit batches.

The arms ran in sequential blocks, not interleaved A/B pairs. Bootstrap intervals resample the 11 processes independently and describe sampling variability only; they do not cover shared-host drift. PMU events were multiplexed and some VM events were unsupported. These limitations do not turn the large observed timing difference into a precise hardware-stall diagnosis.

Counters confirm less container and interpreter work

Event per work unit	Upstream (M)	Optimized (M)	Change
instructions	1691.79	1101.15	-34.91%
cycles	549.83	347.97	-36.71%
L1-dcache-loads	535.85	351.72	-34.36%
branches	193.59	125.50	-35.17%

Native debug evidence	Upstream	flat_pow
All samples	1,175	725
Evaluator self	526	285
GetItem/SetItem inclusive union	206	2
PyFloat_FromDouble inclusive	71	73
Float deallocation inclusive	41	42
libm pow self	20	19

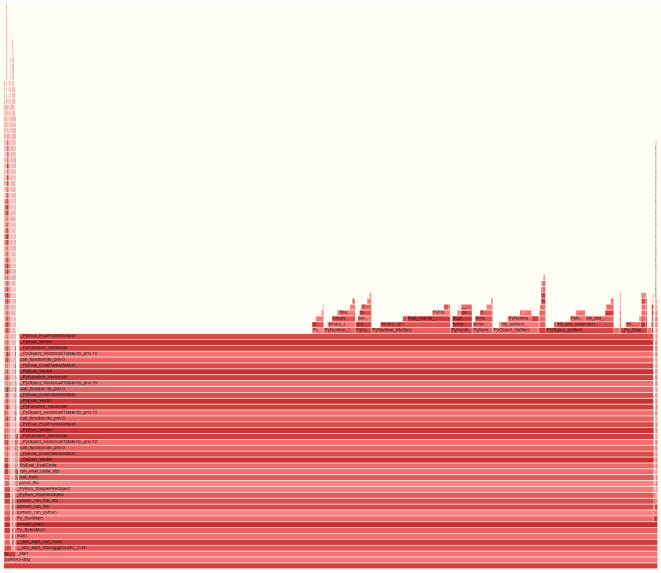
The generic item-access stacks almost disappear, just as the transformation predicts. Instructions fall 34.91% and cycles 36.71%, consistent with the 36.89% clean-runtime reduction. IPC changes from 3.077 to 3.164; the major gain is the reduction in executed work.

Float creation/recycling and power samples remain approximately stable in absolute count. Their share of the smaller profile can rise even when their work has not increased. The same 270 source-level numeric-result expressions per timestep remain; moving state to Python locals does not unbox it or eliminate numerical result objects.

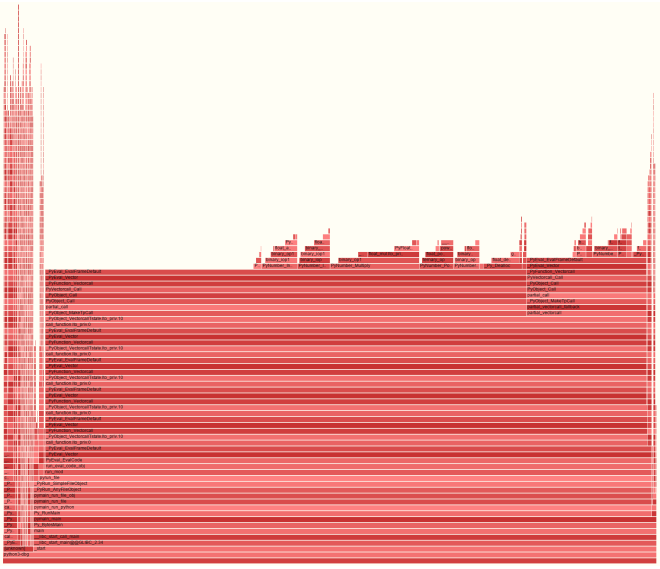
The table deliberately labels self versus inclusive counts. Inclusive categories overlap and must not be added into a runtime partition. Generic cache-event results and L1 miss direction are noisy across separate batches; no cache-miss improvement is needed to explain this result.

Read the remaining flame graph correctly

Upstream: 1,175 samples



flat_pow: 725 samples



Overview of the original native flame graphs. Each graph fills its own width; equal widths do not mean equal runtime. Open the retained SVGs for searchable, zoomable symbol labels.

A flame graph groups sampled call stacks. Width is inclusive sample weight, height is stack depth, and horizontal position is not time. Self samples count only the leaf; inclusive parents overlap. The native profile contains CPython implementation frames, while cProfile supplies Python function names and exact call counts. Together they connect source changes to removed work.

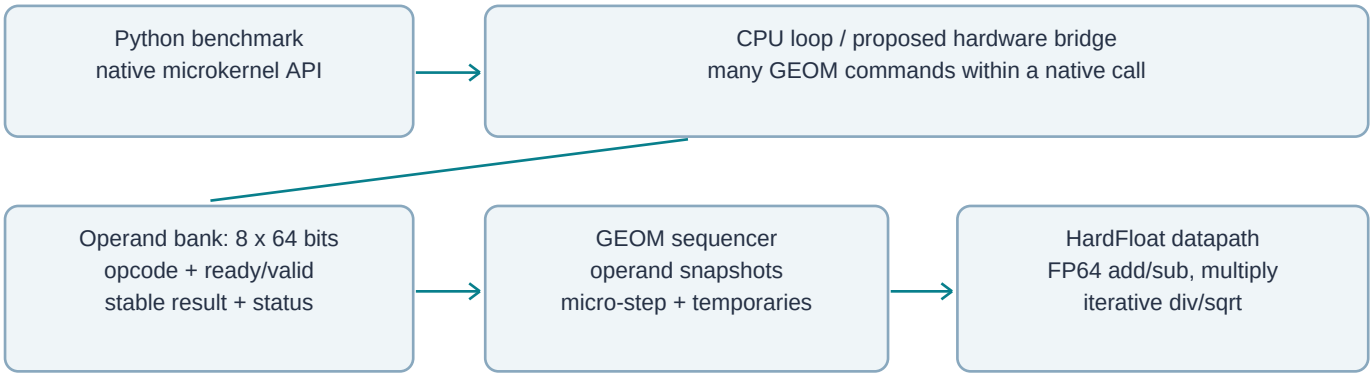
Both versions execute Python arithmetic, so evaluator, numeric dispatch, float handling and pow remain visible. The optimized wrapper introduces its own call ancestry; a stack can become taller while its workload becomes faster. The important evidence is the changed leaf work and the disappearance of item-access paths, not whether the overall silhouette looks shorter.

Residual evaluator self share is 285/725 = 39.31%, not the baseline's 44.77%. This is a debug-build sample fraction, not the fraction of release runtime that a new device could remove. A hardware estimate must count its own requests, cycles, transfers and remaining CPU work.

Native sampling uses equal five-unit work with a fixed 5-million-cycle period. The raw perf data, perf script dump, self/inclusive reports, folded stacks, top-30 stacks, icicle, Python-filtered view and differential flame graphs remain in results/. The Python-filtered native graph is not a source-line Python profiler.

One unit, with raytrace as the primary target

The shared GEOM unit can compute the pairwise squared distance $d^2 = (dx*dx + dy*dy) + dz*dz$ with its VDOT3 command. To preserve the exact final nbody contract, the CPU retains $dt * d^{2**}(-1.5)$, mass factors, ordered velocity updates, position updates and energy evaluation. The final measured kernel remains flat_pow. Moving a whole timestep loop to native C could reduce interpreter and boxed-number overhead, but that would be a separate software control, not evidence of custom hardware acceleration.



Solid datapath boxes describe included RTL; CPU-to-device binding is proposed integration.
Ordered operations round separately. One outstanding command; no FMA or DMA engine.

Architecture and interface. The submitted GEOM unit contains eight 64-bit operand registers, ten 64-bit temporaries, a micro-operation controller, HardFloat add/subtract and multiply logic, and one iterative divide/square-root core. Each operation rounds separately to binary64, round-to-nearest ties-to-even; gradual underflow is retained and FMA is absent. A 50 MHz target is specified, without claiming timing closure. The controller waits for actual arithmetic completion, rather than using behavioral real arithmetic with artificial delays.

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

A shared interface with an exactness boundary

Interface	Contract
Operands	op_we + op_addr[2:0] + op_wdata[63:0], idle writes
Command	cmd_opcode[7:0], cmd_valid / cmd_ready
Response	rsp_result[63:0], rsp_status[3:0], rsp_valid / rsp_ready
Control	clk, active-low rst_n; one command outstanding
Status	OK, ROOT_NONE, NONFINITE, BADOP

The host finishes writes before issuing a command. Writes while busy are ignored; a response remains stable until consumed. Reset aborts in-flight work. The CPU retains traversal, original error behavior, and a fallback for exceptional results. The proposed stack is benchmark → native adapter → processor bridge → GEOM. A CPU-native numerical control is included, but no deployed processor bridge or custom instruction is claimed. Moving a loop into C and accelerating it in hardware are separate contributions.

Exactness. VDOT3 uses three separately rounded multiplications followed by two ordered additions, matching the selected distance expression. The optional RMAG command instead computes $dt / (d^2 * \mathrm{sqrt}(d^2))$; it corresponds to flat_sqrt and can alter the accumulated trajectory. RMAG is included as an explicitly approximate capability, not a replacement for the bit-identical submitted kernel. Neither energy tolerance alone nor expression tests establish exact final state equivalence.

Area and power trade-offs: sharing arithmetic limits duplicated logic and register storage, but serialization can extend active time. Holding unused operands stable reduces switching; a clock-enable or gated implementation could save idle power, at an implementation cost. More lanes or deeper pipelining could raise throughput while adding registers, clock load and routing. Lower instantaneous power does not guarantee lower energy per frame. These are design trade-offs, not numerical power or area results.

Interactive: hierarchical block diagram

Interactive: command and datapath flow

Interactive: complete workload stream

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

Why we do not claim a nbody hardware speedup

Schedule and feasibility. A VDOT3 command needs six operand writes, one command cycle, ten micro-operation cycles, and one response-retirement cycle: 18 cycles in the conservative transaction budget. There are ten pairs per step and 20,000 steps per unit, hence 200,000 hardware commands even if the Python/native boundary is crossed only once. At 50 MHz that is 72 ms of device work. An illustrative additional 0.10 microseconds per command adds 20 ms, so total offload cost is 92 ms.

Compared with the measured 143.3736875 ms/unit flat_pow runtime, $T_{est} = 143.3736875(1 - f) + 92$ ms. Breaking even requires removal of more than 64.17% of runtime. The profiles do not establish such a removable share for the dot product alone. Therefore the shared operation is logically applicable to exact nbody, but this design does not claim a nbody hardware speedup. Raytrace is the primary acceleration target; the project proposal need not force one unit to improve both workloads.

Trade-offs and validation. One shared adder, multiplier and iterative divide/square-root unit saves duplication but serializes work. A whole-system resident-state engine or a lower-cost processor instruction could amortize communication further, at the cost of a different design and a new estimate. The submitted design is deliberately bounded. Host reference checks and RTL semantic elaboration pass; no RTL simulation, synthesis, area/power extraction or physical timing is claimed. The numerical default and the software speedup measurements are unchanged by this proposal.

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

What the final submission establishes

flat_pow keeps the five-body state local across all 20,000 steps, eliminating repeated container work while preserving the original arithmetic and update order. The latest matched pair gives 36.89% less runtime with identical energy and all state bits. grouped was a useful intermediate; the decisive change was extending state reuse across the whole advance call.

The shared GEOM proposal supplies an explicit hardware datapath, controller and numerical interface. Its estimated benefit is separated from measured software gains, and native CPU execution is separated from a proposed device bridge. The hardware documents state validation coverage and area/power/frequency assumptions; no measured hardware speedup or achieved clock is claimed.

```
./script_nbody.sh --variant both --loops 16
LOOPS=16 ./tools/ab_timing.sh nbody 16
```

Deliverable	Location
Required report and runner	report_nbody.txt; script_nbody.sh
PDF and reviewed sources	docs/reports/report_nbody.pdf docs/reports/nbody_source.json
Selected raw evidence	results/latest_nbody_baseline results/latest_nbody_optimized
Comparison and analysis	results/latest_comparison_nbody analysis/latest_results_analysis.json
HDL, software interface and diagrams	hardware/docs/HARDWARE_DESIGN.md hardware/diagrams/
Prompt record and requirements mapping	prompt.txt docs/SUBMISSION_CHECKLIST.md

Sources: user-supplied report drafts and hardware notes (hashes in docs/provenance/source_inputs.json); the selected raw runs; byte-verified pyperformance 1.14.0 sources; CPython-native perf profiles. This report supersedes the supplied draft's default and unsupported hardware claims. Software measurements remain original VM evidence.